

PZ-NRF24L01 模块开发手册

1.1 NRF24L01 无线模块简介

NRF24L01 无线模块，采用的芯片是 NRF24L01，该芯片的主要特点如下：

- 1) 2.4G 全球开放的 ISM 频段，免许可证使用。
- 2) 最高工作速率 2Mbps，高校的 GFSK 调制，抗干扰能力强。
- 3) 125 个可选的频道，满足多点通信和调频通信的需要。
- 4) 内置 CRC 检错和点对多点的通信地址控制。
- 5) 低工作电压（1.9-3.6V）。
- 6) 可设置自动应答，确保数据可靠传输。

该芯片通过 SPI 与外部 MCU 通信，最大的 SPI 速度可以达到 10Mhz，所以在后面软件编程的时候 SPI 速度不能高于这个最大值。本章我们用到的模块是深圳云佳科技生产的 NRF24L01，该模块已经被很多公司大量使用，成熟度和稳定性都是相当不错的。该模块的外形和引脚图如图 1.1.1 所示：

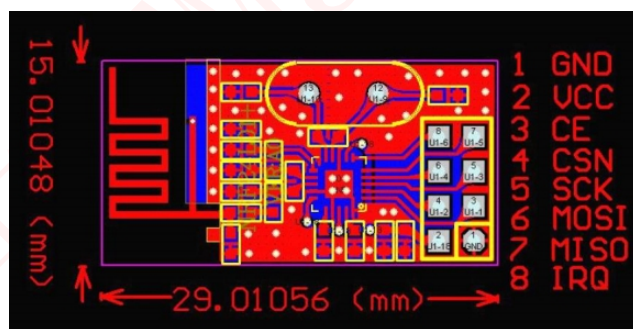


图 1.1.1 NRF24L01 模块外观引脚图

模块 VCC 脚的电压范围为 1.9-3.6V，建议不要超过 3.6V，否则可能烧坏模块，一般用 3.3V 电压比较合适。除了 VCC 和 GND 脚，其他引脚都可以和 5V 单片机的 IO 口直连，正是因为其兼容 5V 单片机的 IO，故使用上具有很大优势。关于 NRF24L01 的详细介绍，请参考 NRF24L01 的技术手册。

1.2 硬件设计

本实验功能简介：开机时系统先检测 NRF24L01 模块是否存在，在检测到

NRF24L01 模块之后，根据 KEY_UP 和 KEY1 按键来决定模块的工作模式，在设定好工作模式之后，就会开发发送/接收数据，同样用 DS0 指示灯来指示程序正在运行。

1.2.1 硬件准备

本实验所需要的硬件资源如下：

- ①F407 最小系统&天马&麒麟开发板 2 个
- ②TFTLCD 模块（没有 TFTLCD 也可使用串口输出）
- ③PZ-NRF24L01 模块 2 个
- ④USB 线 2 条（用于供电和模块与电脑串口调试助手通信）

1.2.2 模块与开发板连接

开发板上并没有集成 NRF24L01 无线模块,而是预留了一个模块接口,所以我们需要知道模块接口与开发板对应的管脚原理图，如图 1.2.1 所示：

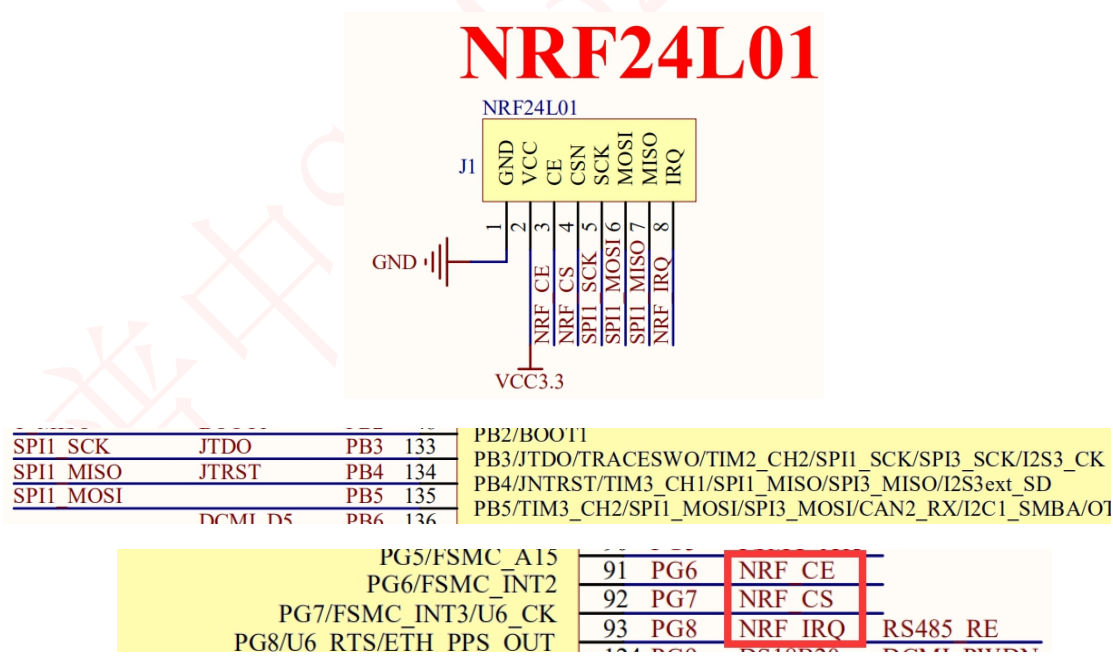
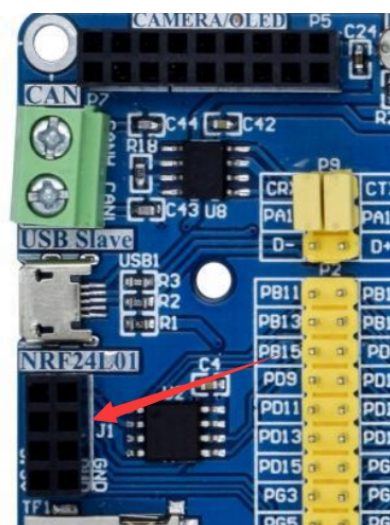


图 1.2.1 NRF24L01 模块接口与开发板连接原理图

这里 NRF24L01 模块使用的是 SPI1，和我们开发板上的 FLASH 共用一个 SPI 接口，所以在使用的时候要分时复用 SPI1。本章我们需要把 FLASH EN25QXX 的片选信号置高，以防止这个器件对 NRF24L01 的通信造成干扰。

开发板上 NRF24L01 无线模块接口图如下图所示：



由于 2.4G 无线通信是双向的，所以至少要有两个模块同时能工作，这里我们使用 2 套 STM32 开发板来向大家演示。

1.3 软件设计

打开实验工程，可以看到我们加入了 nrf24l01.c 源文件和 nrf24l01.h 头文件，所有 NRF24L01 相关的驱动代码和定义都在这两个文件中实现。同时，我们还加入了之前的 spi 驱动文件 spi.c 和 spi.h 头文件，因为 NRF24L01 是通过 SPI 接口通信的。

1.3.1 NRF24L01 驱动程序

打开 nrf24l01.c 文件，代码如下：

```
1.  #include "nrf24l01.h"
2.  #include "spi.h"
3.
4.
5.  const u8 TX_ADDRESS[TX_ADR_WIDTH]={0x34,0x43,0x10,0x10,0x01}; //发送地址
6.  const u8 RX_ADDRESS[RX_ADR_WIDTH]={0x34,0x43,0x10,0x10,0x01};
7.
8.  //初始化 24L01 的 IO 口
9.  void NRF24L01_Init(void)
10. {
11.     GPIO_InitTypeDef GPIO_InitStructure;
12.     SPI_InitTypeDef SPI_InitStructure;
```

```

13.
14.      RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOG, ENABLE);    //使能 PB,G
      端口时钟
15.
16.
17.      GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12;    //PB12 上拉 防止 W25X 的干扰
18.      GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;    //推挽输出
19.      GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
20.      GPIO_Init(GPIOB, &GPIO_InitStructure);    //初始化指定 IO
21.      GPIO_SetBits(GPIOB,GPIO_Pin_12);//上拉
22.
23.
24.      GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7|GPIO_Pin_8;    //PG8 7 推挽
25.      GPIO_Init(GPIOG, &GPIO_InitStructure);//初始化指定 IO
26.
27.      GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
28.      GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPD;    //PG6 输入
29.      GPIO_Init(GPIOG, &GPIO_InitStructure);
30.
31.      GPIO_ResetBits(GPIOG,GPIO_Pin_6|GPIO_Pin_7|GPIO_Pin_8);//PG6,7,8 上
      拉
32.
33.      SPI2_Init();    //初始化 SPI
34.
35.      SPI_Cmd(SPI2, DISABLE);    // SPI 外设不使能
36.
37.      SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex;    //SPI 设置为双线双向全
      双工
38.      SPI_InitStructure.SPI_Mode = SPI_Mode_Master;    //SPI 主机
39.      SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;    //发送接收 8 位帧结构
40.      SPI_InitStructure.SPI_CPOL = SPI_CPOL_Low;    //时钟悬空低
41.      SPI_InitStructure.SPI_CPHA = SPI_CPHA_1Edge;    //数据捕获于第 1 个时钟沿
42.      SPI_InitStructure.SPI_NSS = SPI_NSS_Soft;    //NSS 信号由软件控制
43.      SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_16;    //定义波特率预分
      频的值:波特率预分频值为 16
44.      SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;    //数据传输从 MSB 位开始
45.      SPI_InitStructure.SPI_CRCPolynomial = 7;    //CRC 值计算的多项式
46.      SPI_Init(SPI2, &SPI_InitStructure);    //根据 SPI_InitStruct 中指定的参数初始化外设 SPIx 寄存
      器
47.
48.      SPI_Cmd(SPI2, ENABLE);    //使能 SPI 外设
49.
50.      NRF24L01_CE=0;    //使能 24L01
51.      NRF24L01_CSN=1;    //SPI 片选取消

```

```

52. }
53.
54. //检测 24L01 是否存在
55. //返回值:0, 成功;1, 失败
56. u8 NRF24L01_Check(void)
57. {
58.     u8 buf[5]={0XA5,0XA5,0XA5,0XA5,0XA5};
59.     u8 i;
60.     SPI2_SetSpeed(SPI_BaudRatePrescaler_4); //spi 速度为 9Mhz ( 24L01 的最大 SPI 时钟为
        10Mhz )
61.     NRF24L01_Write_Buf(NRF_WRITE_REG+TX_ADDR,buf,5); //写入 5 个字节的地址.
62.     NRF24L01_Read_Buf(TX_ADDR,buf,5); //读出写入的地址
63.     for(i=0;i<5;i++)if(buf[i]!=0XA5)break;
64.     if(i!=5)return 1; //检测 24L01 错误
65.     return 0;        //检测到 24L01
66. }
67.
68. //SPI 写寄存器
69. //reg:指定寄存器地址
70. //value:写入的值
71. u8 NRF24L01_Write_Reg(u8 reg,u8 value)
72. {
73.     u8 status;
74.     NRF24L01_CSN=0;        //使能 SPI 传输
75.     status =SPI2_ReadWriteByte(reg); //发送寄存器号
76.     SPI2_ReadWriteByte(value);    //写入寄存器的值
77.     NRF24L01_CSN=1;        //禁止 SPI 传输
78.     return(status);        //返回状态值
79. }
80.
81. //读取 SPI 寄存器值
82. //reg:要读的寄存器
83. u8 NRF24L01_Read_Reg(u8 reg)
84. {
85.     u8 reg_val;
86.     NRF24L01_CSN = 0;        //使能 SPI 传输
87.     SPI2_ReadWriteByte(reg); //发送寄存器号
88.     reg_val=SPI2_ReadWriteByte(0XFF); //读取寄存器内容
89.     NRF24L01_CSN = 1;        //禁止 SPI 传输
90.     return(reg_val);        //返回状态值
91. }
92.
93. //在指定位置读出指定长度的数据
94. //reg:寄存器(位置)

```

```

95.  // *pBuf: 数据指针
96.  // len: 数据长度
97.  // 返回值, 此次读到的状态寄存器值
98.  u8 NRF24L01_Read_Buf(u8 reg, u8 *pBuf, u8 len)
99.  {
100.     u8 status, u8_ctr;
101.     NRF24L01_CSN = 0;           // 使能 SPI 传输
102.     status = SPI2_ReadWriteByte(reg); // 发送寄存器值(位置), 并读取状态值
103.     for(u8_ctr=0; u8_ctr<len; u8_ctr++) pBuf[u8_ctr] = SPI2_ReadWriteByte(0xFF); // 读出数据
104.     NRF24L01_CSN = 1;           // 关闭 SPI 传输
105.     return status;              // 返回读到的状态值
106. }
107.
108. // 在指定位置写指定长度的数据
109. // reg: 寄存器(位置)
110. // *pBuf: 数据指针
111. // len: 数据长度
112. // 返回值, 此次读到的状态寄存器值
113. u8 NRF24L01_Write_Buf(u8 reg, u8 *pBuf, u8 len)
114. {
115.     u8 status, u8_ctr;
116.     NRF24L01_CSN = 0;           // 使能 SPI 传输
117.     status = SPI2_ReadWriteByte(reg); // 发送寄存器值(位置), 并读取状态值
118.     for(u8_ctr=0; u8_ctr<len; u8_ctr++) SPI2_ReadWriteByte(*pBuf++); // 写入数据
119.     NRF24L01_CSN = 1;           // 关闭 SPI 传输
120.     return status;              // 返回读到的状态值
121. }
122.
123. // 启动 NRF24L01 发送一次数据
124. // txbuf: 待发送数据首地址
125. // 返回值: 发送完成状况
126. u8 NRF24L01_TxPacket(u8 *txbuf)
127. {
128.     u8 sta;
129.     SPI2_SetSpeed(SPI_BaudRatePrescaler_4); // spi 速度为 9Mhz (24L01 的最大 SPI 时钟为 10Mhz)
130.     NRF24L01_CE = 0;
131.     NRF24L01_Write_Buf(WR_TX_PLOAD, txbuf, TX_PLOAD_WIDTH); // 写数据到 TX BUF 32 个字节
132.     NRF24L01_CE = 1; // 启动发送
133.     while(NRF24L01_IRQ != 0); // 等待发送完成
134.     sta = NRF24L01_Read_Reg(STATUS); // 读取状态寄存器的值
135.     NRF24L01_Write_Reg(NRF_WRITE_REG + STATUS, sta); // 清除 TX_DS 或 MAX_RT 中断标志
136.     if(sta & MAX_TX) // 达到最大重发次数
137.     {
138.         NRF24L01_Write_Reg(FLUSH_TX, 0xFF); // 清除 TX FIFO 寄存器

```

```

139.         return MAX_TX;
140.     }
141.     if(sta&TX_OK)//发送完成
142.     {
143.         return TX_OK;
144.     }
145.     return 0xff;//其他原因发送失败
146. }
147.
148. //启动 NRF24L01 发送一次数据
149. //txbuf:待发送数据首地址
150. //返回值:0, 接收完成; 其他, 错误代码
151. u8 NRF24L01_RxPacket(u8 *rxbuf)
152. {
153.     u8 sta;
154.     SPI2_SetSpeed(SPI_BaudRatePrescaler_8); //spi 速度为 9Mhz(24L01 的最大 SPI 时钟为 10Mhz)
155.     sta=NRF24L01_Read_Reg(STATUS); //读取状态寄存器的值
156.     NRF24L01_Write_Reg(NRF_WRITE_REG+STATUS,sta); //清除 TX_DS 或 MAX_RT 中断标志
157.     if(sta&RX_OK)//接收到数据
158.     {
159.         NRF24L01_Read_Buf(RD_RX_PLOAD,rxbuf,RX_PLOAD_WIDTH); //读取数据
160.         NRF24L01_Write_Reg(FLUSH_RX,0xff); //清除 RX FIFO 寄存器
161.         return 0;
162.     }
163.     return 1; //没收到任何数据
164. }
165.
166. //该函数初始化 NRF24L01 到 RX 模式
167. //设置 RX 地址, 写 RX 数据宽度, 选择 RF 频道, 波特率和 LNA HCURR
168. //当 CE 变高后, 即进入 RX 模式, 并可以接收数据了
169. void NRF24L01_RX_Mode(void)
170. {
171.     NRF24L01_CE=0;
172.     NRF24L01_Write_Buf(NRF_WRITE_REG+RX_ADDR_P0, (u8*)RX_ADDRESS, RX_ADR_WIDTH); //写 RX 节点地址
173.
174.     NRF24L01_Write_Reg(NRF_WRITE_REG+EN_AA, 0x01); //使能通道 0 的自动应答
175.     NRF24L01_Write_Reg(NRF_WRITE_REG+EN_RXADDR, 0x01); //使能通道 0 的接收地址
176.     NRF24L01_Write_Reg(NRF_WRITE_REG+RF_CH, 40); //设置 RF 通信频率
177.     NRF24L01_Write_Reg(NRF_WRITE_REG+RX_PW_P0, RX_PLOAD_WIDTH); //选择通道 0 的有效数据宽度
178.     NRF24L01_Write_Reg(NRF_WRITE_REG+RF_SETUP, 0x0f); //设置 TX 发射参数, 0db 增益, 2Mbps, 低噪声增益
    开启

```

```

179.         NRF24L01_Write_Reg(NRF_WRITE_REG+CONFIG, 0x0f); // 配置基本工作模式的参
           数;PWR_UP,EN_CRC,16BIT_CRC,接收模式
180.         NRF24L01_CE = 1; //CE 为高,进入接收模式
181.     }
182.
183. //该函数初始化 NRF24L01 到 TX 模式
184. //设置 TX 地址,写 TX 数据宽度,设置 RX 自动应答的地址,填充 TX 发送数据,选择 RF 频道,波特率和 LNA HCURR
185. //PWR_UP,CRC 使能
186. //当 CE 变高后,即进入 RX 模式,并可以接收数据了
187. //CE 为高大于 10us,则启动发送.
188. void NRF24L01_TX_Mode(void)
189. {
190.     NRF24L01_CE=0;
191.     NRF24L01_Write_Buf(NRF_WRITE_REG+TX_ADDR,(u8*)TX_ADDRESS,TX_ADR_WIDTH); //写 TX 节点地址
192.     NRF24L01_Write_Buf(NRF_WRITE_REG+RX_ADDR_P0,(u8*)RX_ADDRESS,RX_ADR_WIDTH); //设置 TX 节点
           地址,主要为了使能 ACK
193.
194.     NRF24L01_Write_Reg(NRF_WRITE_REG+EN_AA,0x01); //使能通道 0 的自动应答
195.     NRF24L01_Write_Reg(NRF_WRITE_REG+EN_RXADDR,0x01); //使能通道 0 的接收地址
196.     NRF24L01_Write_Reg(NRF_WRITE_REG+SETUP_RETR,0x1a); //设置自动重发间隔时间:500us + 86us;最大
           自动重发次数:10 次
197.     NRF24L01_Write_Reg(NRF_WRITE_REG+RF_CH,40); //设置 RF 通道为 40
198.     NRF24L01_Write_Reg(NRF_WRITE_REG+RF_SETUP,0x0f); //设置 TX 发射参数,0db 增益,2Mbps,低噪声增
           益开启
199.     NRF24L01_Write_Reg(NRF_WRITE_REG+CONFIG,0x0e); // 配置基本工作模式的参
           数;PWR_UP,EN_CRC,16BIT_CRC,接收模式,开启所有中断
200.     NRF24L01_CE=1; //CE 为高,10us 后启动发送
201. }

```

此部分代码我们不多介绍,程序内有详细的注释,在这里强调一个要注意的地方,在 NRF24L01_Init 函数里面,我们调用了 SPI2_Init() 函数,该函数我们在 FLASH 实验中讲到过,当时我们把 SPI 的 SCK 设置为空闲时为高,但是 NRF24L01 的 SPI 通信时序如图 1.3.1 所示:

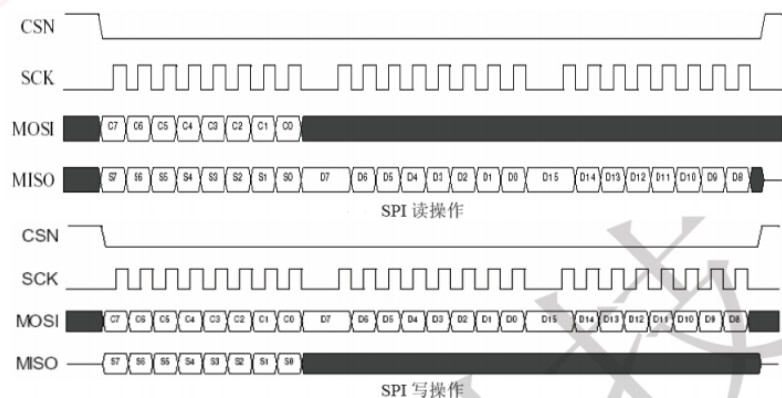


图 1.3.1 NRF24L01 SPI 通信时序图

从图中可以看出，SCK 空闲的时候是低电平的，而数据在 SCK 的上升沿被读写。所以，我们需要设置 SPI 的 CPOL 和 CPHA 均为 0，来满足 NRF24L01 对 SPI 操作的要求。所以，我们在 NRF24L01_Init 函数里面又单独添加了将 CPOL 和 CPHA 设置为 0 的代码。

接下来我们看看 nrf24l01.h 代码，该头文件主要定义了一些 NRF24L01 的命令字以及函数声明，这里还通过 TX_PLOAD_WIDTH 和 RX_PLOAD_WIDTH 决定了发射和接收的数据宽度，也就是我们每次发射和接受的有效字节数。NRF24L01 每次最多传输 32 个字节，再多的字节传输则需要多次传送。

1.3.2 主函数

打开 main.c，代码如下：

```

1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "nrf24l01.h"
8.
9.
10. void data_pros()    //数据处理函数
11. {
12.     u8 key;
13.     static u8 mode=2; //模式选择
14.     u8 rx_buf[33]="www.prechin.cn";
15.     static u16 t=0;
16.     while(1)         //等待按键按下进行选择发送还是接收
17.     {
18.         key=KEY_Scan(0);
19.         if(key==KEY_UP_PRESS)    //接收模式
20.         {
21.             mode=0;
22.             LCD_ShowString(10,140,tftlcd_data.width,tftlcd_data.height,16,"RX_Mode");
23.             LCD_ShowString(10,160,tftlcd_data.width,tftlcd_data.height,16,"Received Data:");
24.             LCD_ShowString(120,160,tftlcd_data.width,tftlcd_data.height,16,"
");

```

```

25.         break;
26.     }
27.     if(key==KEY1_PRESS) //发送模式
28.     {
29.         mode=1;
30.         LCD_ShowString(10,140,tftlcd_data.width,tftlcd_data.height,16,"TX_Mode");
31.         LCD_ShowString(10,160,tftlcd_data.width,tftlcd_data.height,16,"Send Data:  ");
32.         LCD_ShowString(120,160,tftlcd_data.width,tftlcd_data.height,16," ")
        ;
33.         break;
34.     }
35. }
36.
37. if(mode==0) //接收模式
38. {
39.     NRF24L01_RX_Mode();
40.     while(1)
41.     {
42.         if(NRF24L01_RxPacket(rx_buf)==0) //接收到数据显示
43.         {
44.             rx_buf[32]='\0';
45.             LCD_ShowString(120,160,tftlcd_data.width,tftlcd_data.height,16,rx_buf);
46.             break;
47.         }
48.         else
49.         {
50.             delay_ms(1);
51.         }
52.         t++;
53.         if(t==1000)
54.         {
55.             t=0;
56.             LED2=~LED2; //一秒钟改变一次状态
57.         }
58.     }
59. }
60. if(mode==1) //发送模式
61. {
62.     NRF24L01_TX_Mode();
63.     while(1)
64.     {
65.         if(NRF24L01_TxPacket(rx_buf)==TX_OK)
66.         {

```

```

67.         LCD_ShowString(120,160,tftlcd_data.width,tftlcd_data.height,16,rx_buf);
68.         break;
69.     }
70.     else
71.     {
72.         LCD_ShowString(120,160,tftlcd_data.width,tftlcd_data.height,16,"Send Data Failed");
73.     }
74. }
75. }
76. }
77. }
78.
79. int main()
80. {
81.     u8 i=0;
82.
83.     SysTick_Init(168);
84.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
85.     LED_Init();
86.     USART1_Init(115200);
87.     TFTLCD_Init(); //LCD 初始化
88.     KEY_Init();
89.     NRF24L01_Init();
90.
91.
92.     FRONT_COLOR=BLACK;
93.     LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"PRECHIN-STM32F");
94.     LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.net");
95.     LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,16,"NRF24L01 Test");
96.     LCD_ShowString(10,70,tftlcd_data.width,tftlcd_data.height,16,"KEY_UP:RX_Mode KEY1:TX_Mode");
97.     FRONT_COLOR=RED;
98.
99.     while(NRF24L01_Check()) //检测 NRF24L01 是否存在
100.    {
101.        LCD_ShowString(140,50,tftlcd_data.width,tftlcd_data.height,16,"Error");
102.    }
103.    LCD_ShowString(140,50,tftlcd_data.width,tftlcd_data.height,16,"Success");
104.
105.    while(1)
106.    {
107.        data_pros();

```

```

108.         i++;
109.         if(i%20==0)
110.         {
111.             LED1=!LED1;
112.         }
113.
114.         delay_ms(10);
115.     }
116. }

```

程序运行时先通过 NRF24L01_Check 函数检测 NRF24L01 是否存在,如果存在,则让用户选择发送模式(KEY1)还是接收模式(KEY_UP),在确定模式之后,设置 NRF24L01 的工作模式,然后执行相应的数据发送/接收处理。**在测试的时候一定要注意,两块开发板一个选择发送模式,一个选择接收模式,这样在 LCD 上才会显示发送的字符数据“www.prechin.cn”,还有要注意发送字节的长度,在头文件内我们已经定义了最大的发送字节长度。**

1.4 实验现象

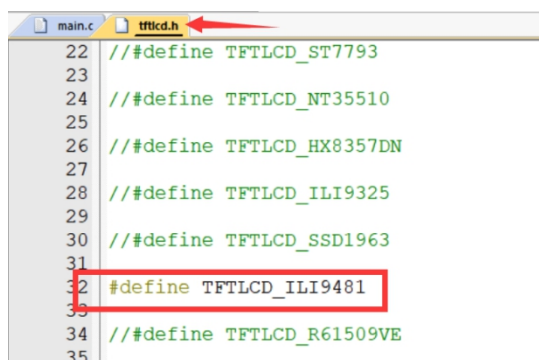
首先,请先确保硬件都已经连接好:

①使用 2 个 PZ-NRF24L01 模块与 2 块 F407 最小系统&天马&麒麟开发板连接(可参考硬件设计小节)

②开发板插上 TFTLCD 液晶(**没有 TFTLCD 的用户可自行修改使用串口输出测试**)。

③使用 USB 线给开发板供电。

注意:如果使用普中 TFTLCD 触摸屏,用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置,比如我使用的 TFTLCD 是 ILI9481,那么就要把这个宏定义放开,其他的注释掉。如下所示:



```
main.c tftlcd.h
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）可以看到 TFTLCD 上显示如下图所示：

将模块连接好以后，把实验程序分别下载到两块开发板内即可看到两块开发板 LCD 显示，插上 NRF24L01 模块后，通过 KEY_UP 和 KEY1 按键，设定好对应的模式，发送端就会发送 `www.prechin.cn` 到接收端，同时 LCD 会显示发送与接收的字符。如图 1.4.1 所示：

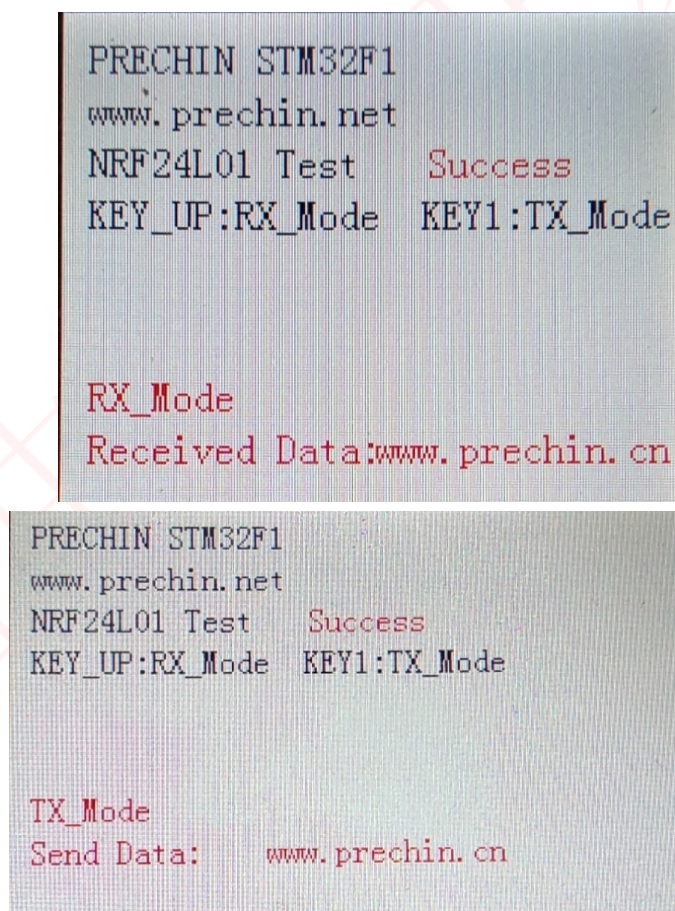


图 1.4.1 实验现象

2 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）